The `AppDisplaySettings` class shown in the previous example could be simple, exposing only the values related to the home page:

```
public class AppDisplaySettings
{
    public string Title { get; set; }
    public bool ShowCopyright { get; set; }
}
```

Your options classes need to be nonabstract and have a `public` parameterless constructor to be eligible for binding. The binder sets any public properties that match configuration values, as you'll see in section 10.3.3.

TIP You're not restricted to built-in types such as `string` and `bool`; you can use nested complex types too. The options system binds sections to complex properties. See the associated source code for examples.

To help facilitate the binding of configuration values to your custom POCO options classes, ASP.NET Core introduces the `IOptions<T>` interface, a simple interface with a single property, `Value`, that contains your configured POCO options class at runtime. Options classes are configured as services in Program.cs , as shown in the following listing.

Listing 10.8 Configuring the options classes using `Configure<T>` in Startup.cs

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.Configure<MapSettings>(
    builder.Configuration.GetSection("MapSettings"));          ❶
builder.Services.Configure<AppDisplaySettings>(
    builder.Configuration.GetSection("AppDisplaySettings"));   ❷
```

❶ Binds the MapSettings section to the POCO options class MapSettings

❷ Binds the AppDisplaySettings section to the POCO options class AppDisplaySettings

TIP You don't have to use the same name for both the section and class, as I do in listing 10.8; it's simply a convention I like to follow. With this convention, you